

Sicurezza del Software

Università degli Studi di Verona
Imbriani Paolo, VR553697
paolo.imbriani@studenti.univr.it
Professor Mila della Preda

Anno Accademico 2026/2027

Indice

1 Vulnerabilità del Software 3
1.1 SQL Injection 4
1.1.1 Come difendersi 4
1.2 Buffer Overflow 4
1.2.1 Come difendersi 4
1.3 Authentication Failure 5
1.3.1 Come difendersi 5
1.4 Software Tampering / Supply Chain Attack 5
1.4.1 Come difendersi 5
2 Cosa è la Computer Security? 5
2.1 Origini 6
3 Software Security 8
3.1 Minaccia 9
3.1.1 Valutazione del rischio 10
3.2 La triade CIA 10
3.3 Principi per il design di sistemi sicuri 11
3.4 Policy di sicurezza 12
4 Autenticazione 13
4.1 Metodi di autenticazione 14
5 Access Control 16
5.1 DAC vs MAC 17
5.1.1 Bell-LaPadula Model (BLP) 18
5.1.2 Biba 19
5.2 Oltre al DAC 19
5.3 Black Box Security 19
5.4 Gray Box 19
5.5 White box security 20
5.5.1 Code Obfuscation 20

1 Vulnerabilità del Software

1.1 SQL Injection

Uno degli attacchi più famosi e pericolosi è l'SQL Injection.Questo tipo di attacco sfrutta le vulnerabilità nelle applicazioni web che interagiscono con un database SQL.

1. L'attaccante inserisce una query SQL malevola in un campo di input dell'applicazione, come un modulo di login o una barra di ricerca.
2. Se l'applicazione non valida correttamente l'input, la query malevola viene eseguita sul database, permettendo all'attaccante di accedere a dati sensibili, modificare o cancellare informazioni.
3. L'attaccante ripete 1 e 2 finché non ottiene le informazioni desiderate.

1.1.1 Come difendersi

I modi per difendersi dall'SQL Injection sono i seguenti:

- Parametrized queries
- Input Validation
- Least Privilege
- Limit error explanation, l'overexplanation degli errori può semplificare la vita all'attaccante
- SW Testing

1.2 Buffer Overflow

Il Buffer Overflow è un'altra vulnerabilità comune che si verifica quando un programma scrive più dati in un buffer di quanto esso possa contenere. Nel momento in cui viene fatto ciò, la memoria del programma viene corrotta (Memory Corruption), portando a:

- Crash / Denial of Service (DOS)
- Remote Code Execution (RCE)
- Bypass Control
- Log deletion
- Information Disclosure
- Shellcode injection
- Control flow hijacking

Il principale obiettivo è compromettere la memoria e il control flow di un programma eseguito e di conseguenza dare potenzialmente problemi all'intero sistema.

1.2.1 Come difendersi

I modi per difendersi da questo attacco sono i seguenti:

- Secure Functions
- Memory safe languages
- Compiler and OS protections
- SW Testing
- Check memory limits

1.3 Authentication Failure

Ci sono diversi tipi di vulnerabilità legate all'autenticazione, tra cui:

- Weak passwords: password deboli o facilmente indovinabili.
- MFA bypass: vulnerabilità che permettono di bypassare l'autenticazione a più fattori.
- Credential stuffing: attacco che sfrutta credenziali rubate da altri servizi per accedere a un account. Questo è il motivo per cui è sconsigliato usare la stessa password per molti servizi e soprattutto usarla per un servizio importante come la banca, università, etc…
- Incorrect Session Management: vulnerabilità nella gestione delle sessioni che possono permettere a un attaccante di impersonare un utente legittimo.
- SQL Injection: vulnerabilità che permette a un attaccante di eseguire query SQL malevole per ottenere accesso non autorizzato.

L'attaccante è capace di entrare all'interno del sistema con le credenziali di una vittima. Vengono rubate le credenziali di un utente legittimo e l'attaccante può accedere al sistema come se fosse quell'utente oppure accedere ai servizi della vittima o «chiudere» la vittima fuori dal proprio account.

1.3.1 Come difendersi

I modi per difendersi da questo attacco sono i seguenti:

- Secure Password
- Anomaly Detection
- MFA
- Unique Password
- Rate limiting

1.4 Software Tampering / Supply Chain Attack

In questo attacco, il codice di un software viene modificato in modo malevolo, spesso durante il processo di sviluppo o distribuzione. L'attaccante può inserire codice dannoso che può compromettere la sicurezza del software e dei dati degli utenti. Spesso, nelle Supply Chain Attack, vengono attaccate librerie all'interno di package manager in modo da compromettere gli utenti che le utilizzano e quindi attacca tutta la supply chain.

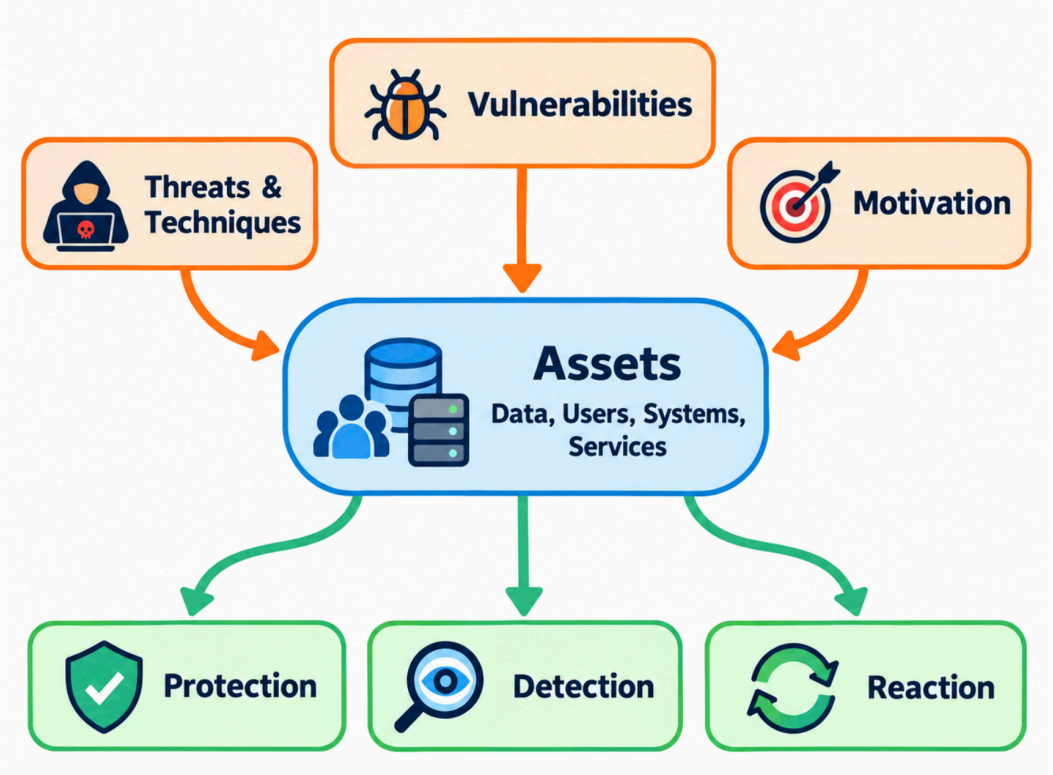
1.4.1 Come difendersi

I modi per difendersi da questo attacco sono i seguenti:

- Monitoring application behaviour
- Code obfuscation
- Integrity check
- Review updates
- Use SBOMs (Software Bill of Materials) and dependencies scanning
- Have an incident-response plan

2 Cosa è la Computer Security?

La sicurezza del software è la disciplina che si occupa di implementare, distribuire e mantenere software che si comporta nella maniera intesa e resiste ad accessi, usi, modifiche non autorizzate.



- L'attaccante vuole ottenere gli asset del sistema (data, utenti, sistemi, servizi)
- Il difensore può proteggersi, riconoscere gli attacchi o reagire in caso di attacco non previsto.

2.1 Origini

La sicurezza informatica nasce negli anni "70, quando i computer erano grandi e costosi e le persone che li utilizzavano erano poche. In quel periodo, la sicurezza era principalmente fisica, con accesso limitato ai computer e alle sale server. Già nel "72, un report chiamato «Computer Secuirty Technology Planning Study» identifica una serie di punti critici sui sistemi che si stavano adottando, come la continua evoluzione dei sistemi tecnologici, la mancanza di standardizzazione e la necessità di proteggere le informazioni sensibili. Nel "76, un altro report, chiamato «Operating Systems Structures to Support Security and Reliable Software», identifica una serie di vulnerabilità legate ai sistemi operativi e propone soluzioni per migliorare la sicurezza dei sistemi informatici.

Computing Surveys, Vol. 8, No. 4, December 1976

Operating System Structures to Support Security and Reliable Software

THEODORE A. LINDEN

Institute for Computer Sciences and Technology, National Bureau of Standards, Washington, D.C. 20234

INTRODUCTION

For the year 1974, one source has identified 339 cases of computer-related crime.[1] The *average* loss in the 339 incidents was $544,000. This average is not distorted by a few exceptional cases; the median loss was very close to the average. Most of the incidents involved simple fraud by an employee who had access to computerized financial records. In 85 percent of the cases, management did not report the incident to the police—often because publicity about it would have been embarrassing.

The fraud is usually possible because of some oversight in an applications system. A simple oversight, for example, may allow a clerk to feed data to an accounts payable system in such a way that no one notices when checks are diverted to a dummy corporation.

If the amount of computer-related fraud is to be controlled, then it is necessary to automate the concepts of segregated duties, independent checking, and accountability for actions that are typical in manual ac-

[1] This information is based on conversations with Robert Courtney. Courtney reports that details on these cases are in his possession, but that they cannot be made public. The work of Parker [PARK 75] based on public reports, supports a similar conclusion about the average loss in computer-related crime.

L'avvento di Internet rompe l'isolamento fisico, all'inizio si pensava ci fosse un problema solo nelle comunicazioni ma ben presto si è capito che il problema era più ampio e riguardava anche i sistemi informatici stessi. **Luciano Floridi** definisce «l'infosfera» come l'ambiente informazionale che consiste in tutti gli oggetti informazionali, le loro relazioni, proprietà, interazioni processi e reazioni. Nell'era dell'AI e degli LLM, è importante interrogarsi su come la sicurezza informatica si evolverà e come le nuove tecnologie impatteranno nella nostra vita. Ora che l'AI è capace di scrivere codice in maniera automatica è importante capire come usarla per difendersi e attaccarsi.

- Gli LLM in difesa, possono trovare vulnerabilità, genera test e usarlo per fare reverse engineering
- Gli LLM in attacco, possono generare codice malevolo, trovare vulnerabilità e generare exploit.



Proteggere i sistemi software come l'autenticazione, access control, vulnerabilità, secure design è una perspettiva complementare all'analizzare e proteggere il software usando tecniche come reverse engineering, offuscazione, watermarking e anti-compromissione.

3 Software Security

La sicurezza non è solo un problema tecnico, ma anche un problema organizzativo e sociale. Le persone coinvolte nello sviluppo, inoltre la responsabilità della sicurezza del software non è solo degli sviluppatori, ma anche dei manager e dei tester. Anche i processi decisi al momento della fase di progettazione devono essere decisivi, decidendo polizze, procedure e incident response. Ovviamente la sicurezza si deve occupare di proteggere gli asset (tutto ciò di valore che deve essere protetto).

Per **temporalità** degli asset, si intende la proprietà degli asset di essere protetti per un certo periodo di tempo, ad esempio, i dati sensibili devono essere protetti per tutta la loro vita utile.

3.1 Minaccia

Una *minaccia* è un impatto negativo indesiderato su un asset.



Figura 1: Minacce e vulnerabilità

Un esempio recente è successo nel settembre 2025, dove PyPi ha subito un attacco di supply chain, compromettendo 35 pacchetti e causando problemi a migliaia di utenti. L'attaccante ha ottenuto accesso a un account di un maintainer manomettendo il codice e abusando la fiducia degli utenti nel pacchetto.

Si può mettere manomettere il codice:

- Manomettendo il comporamento del codice
- Bypassando le licenze

- Piratando il software

Alcuni attacchi come:

- Attacchi di Data Breach (Information Disclosure): In questo attacco vengono rubati dati sensibili di aziende. Nel 2026, IBM ha fatto un report gli attacchi AI-driven sono aumentati del 56%.
- Denial of Service è un attacco che mira a rendere un servizio indisponibile per gli utenti legittimi. Questo può essere fatto sovraccaricando il sistema con richieste, sfruttando vulnerabilità o esaurendo le risorse del server.
- Elevation of Privileges, l'attaccante riesce ad estendere i suoi privilegi a cui ha accesso

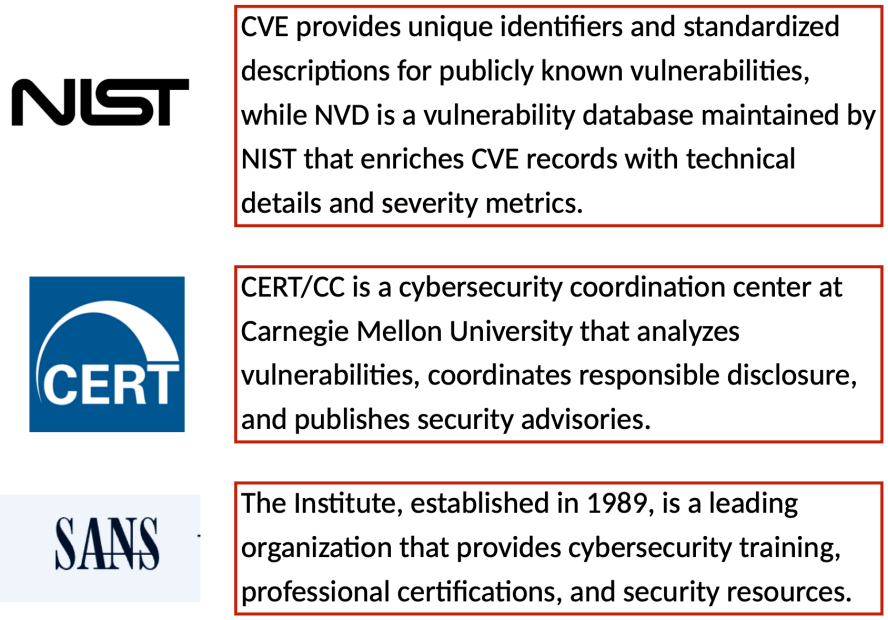


Figura 2: Monitoraggio delle istituzioni di sicurezza informatica

3.1.1 Valutazione del rischio

Bisogna anche valutare il rischio di una minaccia e dare priorità ad essa e capire su quale fronte «spendere» per potersi difendere

- Si produce una lista di minaccie e contromisure
- Si fa una valutazione del rischio e può essere costoso per grandi aziende
- Baseline protection prevede una livello minimo di sicurezza
- Monitoraggio continuo per rilevare nuove minacce e vulnerabilità
- Analisi di asset, minaccie e vulnerabilità che ci permette di scegliere le giuste contromisure.

Le misure di protezione seguono questo workflow:

$$\text{Protezione} \rightarrow \text{Rilevare} \rightarrow \text{Recuperare}$$

3.2 La triade CIA

Ci sono tre principi fondamentali della sicurezza informatica, noti come la triade CIA:

- **Confidenzialità (Confidentiality)**: Garantire che le informazioni siano accessibili solo a persone autorizzate.

- **Integrità (Integrity)**: Garantire che le informazioni siano accurate e complete, e che non siano state alterate in modo non autorizzato.
- **Disponibilità (Availability)**: Garantire che le informazioni e i sistemi siano disponibili quando necessario.

La sicurezza sta nell'intersezione di queste misure. Oltre a questi principali caratteristiche ne esistono altre come:

- **L'accountability**: non possiamo dire con certezza che la sicurezza di un specifico sistema sia impenetrabile e quindi l'accountability riesce a tracciare le azioni degli utenti e dei sistemi, permettendo di identificare eventuali violazioni della sicurezza e di attribuire la responsabilità.
- **L'autenticità**: che ci garantisce che le informazioni sono affidabile, genuine e verificabili. Per un esempio che verifica se un utente è veramente ciò che dice di essere.
- **Non repudiazione**: garantisce che una parte non possa negare l'autenticità di un'azione o di un messaggio inviato. Ad esempio, se un utente invia un messaggio firmato digitalmente, non può successivamente negare di averlo inviato.

3.3 Principi per il design di sistemi sicuri

Ci sono dei common-sense principles per il design di sistemi sicuri, che sono i seguenti:

- I meccanismi di sicurezza dovrebbero essere *più semplici e piccoli possibili* in modo da essere capire, testare e verificarlo. Come per esempio utilizzare un piccolo ma ben testato modulo di autenticazione piuttosto che un complesso sistema di autenticazione fatto ad hoc.
- Gli errori dovrebbe avere **Fail Safe Defaults** ovvero sistemi che nel momento in cui accade un errore, il sistema dovrebbe entrare in uno stato sicuro e non permettere accessi non autorizzati.
- **Complete Mediation**, è un principio di design che dice che ogni accesso ad una risorsa protetta deve essere verificato. I permessi possono cambiare dopo il primo accesso, quindi ogni accesso deve essere controllato.
- La sicurezza di un meccanismo non dove dipendere sulla segretezza del suo design o della sua implementazione, ma dovrebbe essere sicuro anche se il design e l'implementazione sono pubblici. Questo principio è noto come **Open Design**.
- Le azioni critiche devono richiedere il coinvolgimento da un altro soggetto, così che in questo modo l'attaccante dovrebbe attaccare due soggetti per compiere un'azione non desiderata. Questo si chiama **Separation of Duties**.
- **Least Privilege**: Gli utenti e processi dovrebbero ricevere solo i permessi di cui hanno bisogno per svolgere le loro funzioni. Questo riduce il rischio di accessi non autorizzati e limitare i danni in caso di compromissione.
- **Defense in depth**: La difesa in profondità usa multiple e independenti misure di sicurezza così che il crollo di un layer non compromette l'intero sistema.
- **Isolare** è la forma più semplice di protezione. Limitare il numero di sistemi su cui vi sono informazioni critiche, isolandoli *fisicamente* o *logicamente*.
- I meccanismi di sicurezza dovrebbero essere semplici da capire e usarli. Il sistema di sicurezza **non deve essere d'intralcio** alla sicurezza stessa e deve essere ragionevole e minimal.

3.4 Policy di sicurezza

La security policy ci dice:

- Cosa deve essere protetto
- Come questa protezione è implementata
- Se la protezione funziona

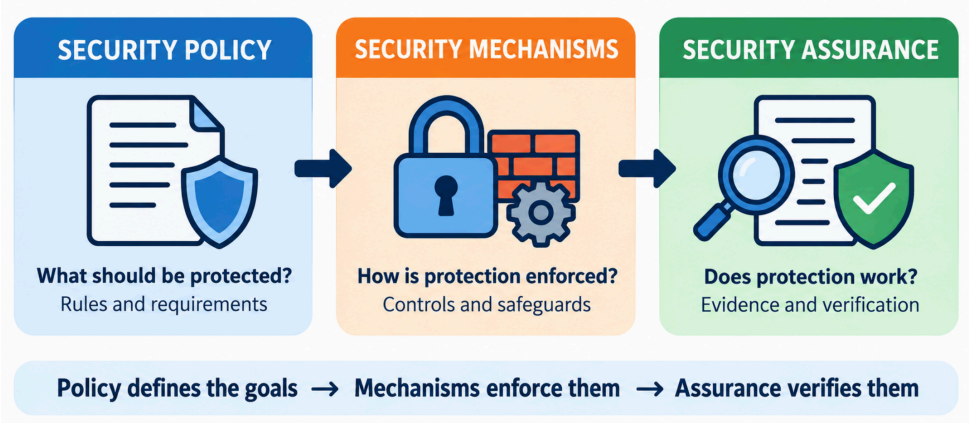


Figura 3: Policy di sicurezza informatica

Una policy di sicurezza caratterizza:

- comportamenti accettabili del sistema
- comportamenti inaccettabili del sistema
- utenti e azioni autorizzate
- condizioni in cui l'accesso è consentito

Una buona polizza dovrebbe essere:

- chiaramente definita
- coerente
- consistente
- implementabile
- verificabile

Le scelte di sicurezza coinvolgono anche trade-offs.

Per astrarre la sicurezza abbiamo bisogno di un modello di sicurezza, che è una rappresentazione formale della security policy. Un modello di sicurezza definisce le regole e le restrizioni per l'accesso alle risorse del sistema, basandosi sulla security policy. I modelli di sicurezza possono essere utilizzati per analizzare e verificare la sicurezza di un sistema, identificare vulnerabilità e progettare contromisure. Il modello rappresenta comportamenti possibili, la policy vincola questi comportamenti in modo da evitarre stati non sicuri.

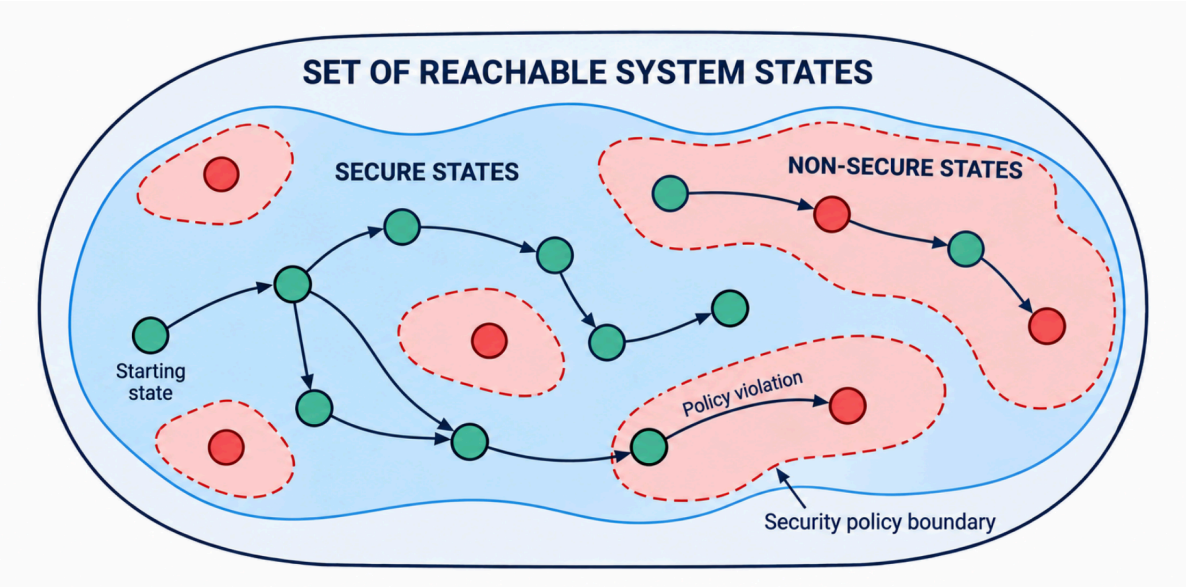


Figura 4: Stati sicuri e non sicuri in un modello di sicurezza

Questo modello *è solo un astrazione e non è una garanzia di sicurezza*, ma ci permette di ragionare sulla sicurezza e di fare delle analisi formali.

Ci dobbiamo anche assicurare che il sistema sia sicuro in ogni momento, quindi dobbiamo fare un'analisi continua e iterativa della sicurezza. Questo processo è noto come PDCA (Plan-Do-Check-Act), che ci permette di pianificare, implementare, verificare e migliorare continuamente la sicurezza del sistema.

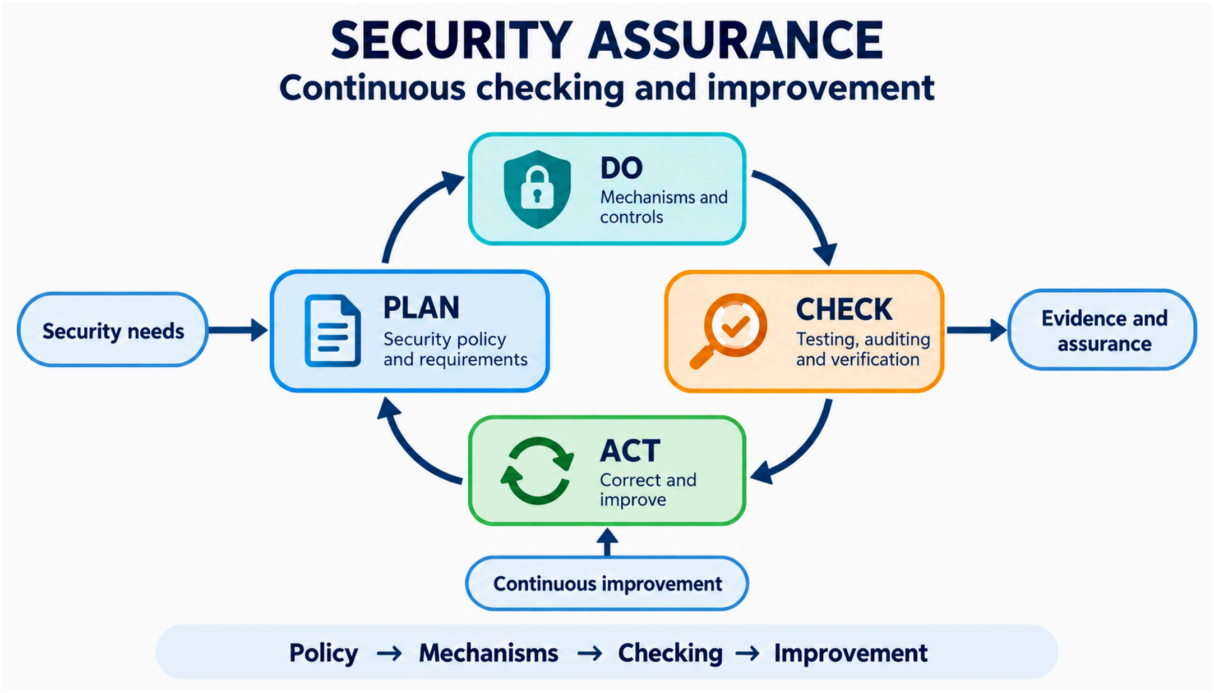


Figura 5: Ciclo PDCA per la sicurezza informatica

4 Autenticazione

L'autenticazione è il modo in cui un sistema verifica l'identità di un utente o di un'entità e quindi serve per proteggere l'identità digitale di un utente in maniera tale da evitare che un attaccante possa impersonare un utente legittimo. L'autenticazione è un processo fondamentale per garantire la sicurezza dei sistemi informatici e proteggere le informazioni sensibili.

L'autenticazione da sola non garantisce l'accountability. Le credenziali rubate possono comunque causare l'attribuzione di un utente legittimo. **MFA, logs protetti e monitoraggio delle sessioni** possono aiutare a garantire l'accountability.



Figura 6: Modi per autenticarsi in un sistema informatico

4.1 Metodi di autenticazione

Solitamente, quando si parla di Password (come metodo di autenticazione) viene fatta attraverso Single Factor Authentication o Multi Factor Authentication (MFA). La MFA è un metodo di autenticazione che richiede agli utenti di fornire due o più prove di identità, note come fattori, per accedere a un sistema o a un servizio.

L'insieme di tutte le possibili password che un attacker dovrebbe provare sono:

$$\text{PS} = |A|^n$$

dove n è la lunghezza della password e A è l'insieme di tutti i caratteri possibili. Sembra un numero impossibile da poter farci brute-force ma ci sono alcune vulnerabilità come:

- Il riuso delle password
- Password prevedibili

Quindi quando creaimo una password dobbiamo fare attenzione a:

- La lunghezza
- L'entropia della password (quanto è casuale)

L'insieme di queste due cose rende la password più forte.

Mirai botnet riuscì a compromettere più di 600.000 dispositivi IoT sfruttando password deboli e prevedibili, causando un attacco DDoS su larga scala. Questo attacco ha evidenziato l'importanza di utilizzare password forti e uniche per ogni dispositivo e servizio.

Ci sono diversi modi per attaccare le password:

- Online, quindi un interazione diretta del servizio, e possiamo difenderci usando rate limiting, throttling, MFA. Gli attacchi possono avvenire attraverso
  - brute-force
  - dictionary attack
  - credential stuffing
  - password spraying
- Offline, tramite l'attaccante che ruba le hashed passwords dal server. Di solito per proteggersi, si utilizza un meccanismo chiamato salt value ovvero un valore casuale aggiunto alla password prima di essere hashata, in modo da rendere più difficile per gli attaccanti utilizzare tabelle pre-computate (rainbow tables) per decifrare le password.

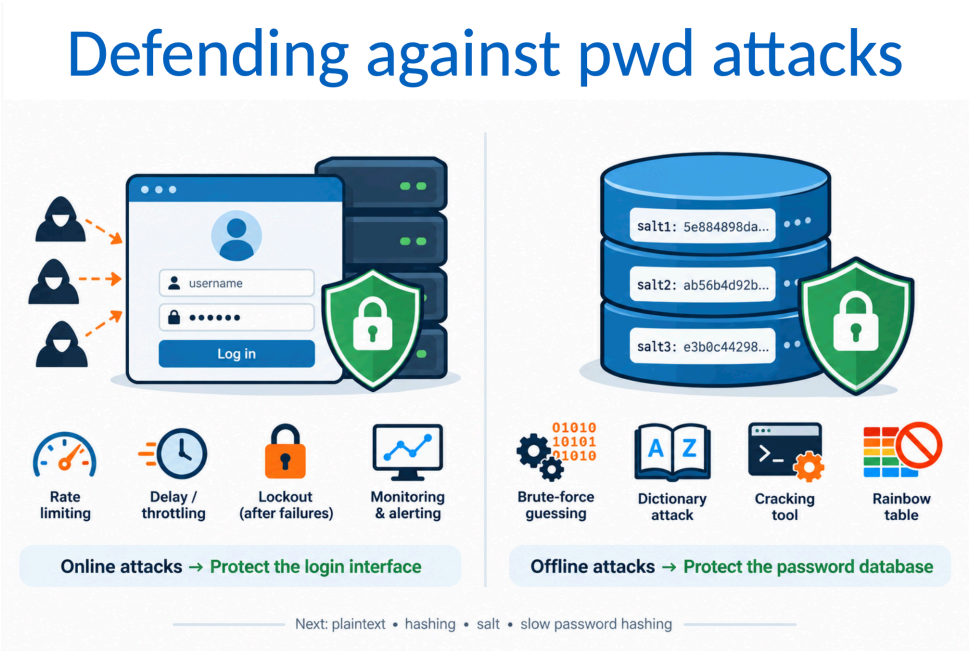


Figura 7: Modi per attaccare le password: online e offline

Alternative alle password sono:

- **Qualcosa che possiedi**: come i token, telefono o smart card
- **Una chiave crittografica**: una chiave privata in un altro dispositivo
- **One time proof**: come i codici generati da app di autenticazione o inviati via SMS, che sono validi solo per un breve periodo di tempo.
- **Autenticazione certificata**: accesso VPN, sistemi aziendali, smart cards, autenticazione dei dispositivi, esercizi sicuri e infrastrutturali. La chiave privata dimostra il possesso senza rivelaren la chiave in sè.
- **Passkey**: è un metodo di autenticazione che utilizza una coppia di chiavi crittografiche per autenticare un utente senza la necessità di una password. La chiave pubblica viene memorizzata sul server, mentre la chiave privata rimane sul dispositivo dell'utente. Quando l'utente tenta di accedere, il server invia una sfida crittografica che può essere risolta solo con la chiave privata dell'utente, garantendo così l'autenticazione senza trasmettere la password.

- **Biometric authorization**: come impronte digitali, riconoscimento facciale o scansione dell'iride, che utilizza caratteristiche fisiche uniche dell'utente per verificare la sua identità.
  Requisiti per autenticazione biometrica:
  - tutte le persone dovrebbero avere questa caratteristica
  - le persone dovrebbero avere differenza in questa caratteristica
  - la caratteristica non deve cambiare troppo durante il tempo
  - la caratteristica deve avere l'abilità di essere identificata

  Le caratteristiche biometriche non sono segreti perfetti:
  - Spoofing: impronta digitale false, fotografie, etc.
  - Privacy: le informazioni biometriche sono molto personali
  - Irreversability: una volta compromessa, non può essere cambiata
  - Template protection: template immagazzinati devono essere protetti
  - Falsi positivi: ci sono dei casi in cui l'utente legittimo non viene riconosciuto oppure un impostore viene riconosciuto.
  - Metodi di recupero possono poi diventare una vulnerabilità

  I dati biometrici sono conveniente ma devono essere protetti.

5 Access Control

L'access control ci permette di limitare l'accesso alle risorse del sistema solo agli utenti autorizzati.

1. L'utente entra nel sistema con successo attraverso *autenticazione*
2. L'utente può accedere solo alle risorse per cui ha i permessi (*autorizzazione*). Il reference monitor fa da «middleman» tra l'utente e la risorsa, controllando se l'utente ha i permessi per accedere alla risorsa richiesta attraverso un database di autorizzazioni
3. Se l'utente cambia l'identificatore della risorsa, il sistema deve negare l'accesso.

I concetti fondamentali dell'access control sono:

- Soggetto: chi richiede l'accesso
- Azioni: leggere, scrivere, eseguire, eliminare, etc…
- Oggetto: quale risorsa viene richiesta
- Decisione della policy: valutare la Policy (quindi la specifica delle regole che definiscono il controllo degli accessi)
- Risultato: accesso negato o concesso

$$\text{Soggetto} + \text{Azioni} + \text{Oggetto} = \text{Access Control}$$

Il reference monitor viene articolato in due componenti:

- **Policy decision point**: prende la decisione di accesso basata sulla policy e sui permessi dell'utente.
- **Policy enforcement point**: applica la decisione di accesso, permettendo o negando l'accesso alla risorsa richiesta.

Il reference monitor deve rispettare tre caratteristiche:

- **Complete mediation**: ogni accesso deve essere controllato, anche se l'utente ha già avuto accesso in precedenza.
- **Tamper-proof**: il reference monitor deve essere protetto da modifiche non autorizzate.
- **Verifiable**: deve essere possibile verificare che il reference monitor funzioni correttamente e rispetti le regole di accesso definite nella policy.

5.1 DAC vs MAC

Ci sono due principali modelli di access control: Discretionary Access Control (DAC) e Mandatory Access Control (MAC).

- **Discretionary Access Control (DAC)**: Il proprietario della risorsa ha il controllo discrezionale su chi può accedere alla risorsa. Gli utenti possono concedere o revocare l'accesso ad altri utenti a loro discrezione. Questo modello è flessibile ma può essere meno sicuro, poiché gli utenti potrebbero concedere accesso a persone non autorizzate.
- **Mandatory Access Control (MAC)**: In questo modello, l'accesso alle risorse è controllato da regole di sicurezza definite dall'amministratore del sistema. Gli utenti non hanno il controllo discrezionale sull'accesso alle risorse. Questo modello è più sicuro, poiché le regole di accesso sono rigorose e non possono essere modificate dagli utenti. L'accesso è controllato da etichette di sicurezza: i soggetti hanno un livello di sicurezza e ogni risorsa classificata. È spesso definito con *multi-level security*.

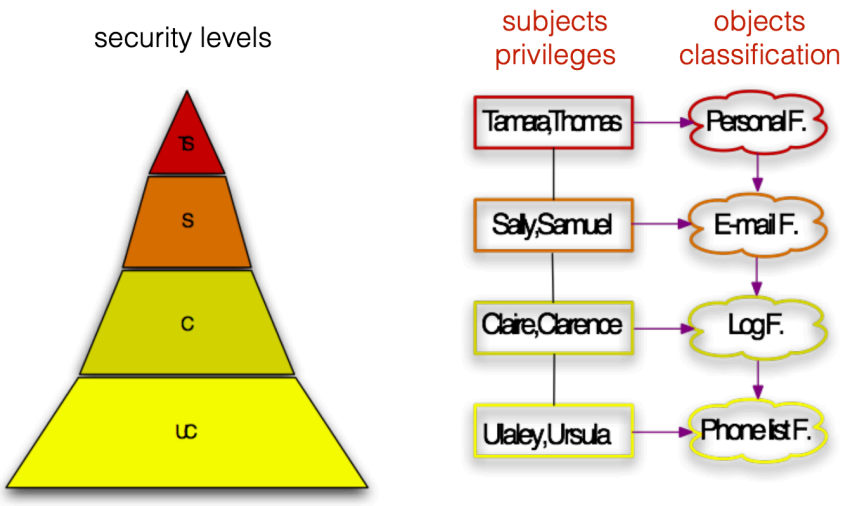


Figura 8: Modello di access control Mandatory Access Control (MAC) con multi-level security

Le informazioni classificate sono associati ad uno o più *compartimenti* che descrive i soggetti.

L'access control può essere visto come Accesso Control Matrix, dove le righe rappresentano i soggetti, le colonne rappresentano gli oggetti e le celle contengono i permessi di accesso. Questo modello permette di visualizzare chiaramente chi ha accesso a cosa e con quali permessi.

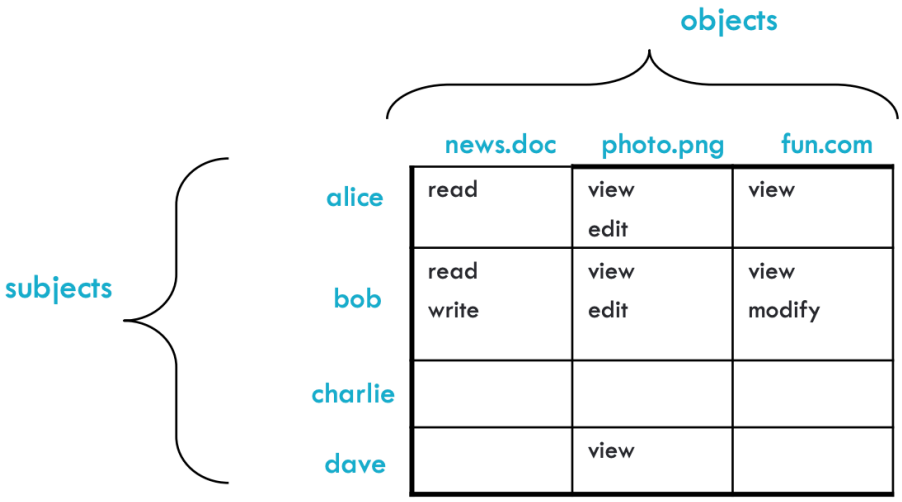


Figura 9: Access Control Matrix

5.1.1 Bell-LaPadula Model (BLP)

Bell-LaPadula è un modello di sicurezza informatica sviluppato negli anni "70 per garantire la confidenzialità delle informazioni in sistemi multi-livello. Il modello si basa su due principi fondamentali: il principio del «no read up» (NRU) e il principio del «no write down» (NWD).

- **No Read Up (NRU)**: Un soggetto con un livello di sicurezza più basso non può leggere informazioni da un oggetto con un livello di sicurezza più alto. Questo principio garantisce che le informazioni sensibili non vengano divulgate a soggetti non autorizzati.
- **No Write Down (NWD)**: Un soggetto con un livello di sicurezza più alto non può scrivere informazioni in un oggetto con un livello di sicurezza più basso.

5.1.2 Biba

Biba invece è un modello di sicurezza informatica sviluppato per garantire l'integrità delle informazioni. Oggetti e soggetti fidati non devono essere contaminati. Cerca di prevenire i dati da essere modificati che ha i ruoli invertiti di BLP ovvero:

- **No Read Down (NRD)**: Un soggetto con un livello di sicurezza più alto non può leggere informazioni da un oggetto con un livello di sicurezza più basso. Questo principio garantisce che le informazioni sensibili non vengano lette da soggetti non autorizzati.
- **No Write Up (NWU)**: Un soggetto con un livello di sicurezza più basso non può scrivere informazioni in un oggetto con un livello di sicurezza più alto. Questo principio garantisce che le informazioni sensibili non vengano modificate da soggetti non autorizzati.

5.2 Oltre al DAC

Ci sono alcuni limiti dovuti al DAC:

- Il numero di utenti aumenta
- Gli utenti cambiano le responsabilità
- I permessi devono essere frequentemente aggiornati
- Gli accessi dipendono dal contesto
- Le risorse siono condivise tra team o progetti

Esistono altri tre modelli:

- **Role Based Access Control (RBAC)**: gli accessi non vengono dati dall'identità dell'utente ma dal ruolo. Il ruolo dirà quello che può fare. Sfruttano il Least Privilege e quindi gli vengono dati permessi per fare svolgere solo le azioni del ruolo.
- **Attribute Based Access Control (ABAC)**: Non guarda i ruoli ma guarda gli attributi. Prendiamo per un esempio una porta con uno smart lock. Vorrei che la baby sitter possa entrare all'interno della casa solo in un certo orario o sfruttando altri environment; Oppure non poter accedere a documenti sensibili da fuori la rete aziendale.
- **Relationship Based Access Control (ReBAC)**: È la relazione con il soggetto proprietario della risorsa cosa io posso o non posso vedere. Un project manager può accettare o rifiutare persone che si possono connettere alla risorsa.

Nella sicurezza, bisogna comunque applicare il *zero-trust* che dice che non bisognerebbe mai fidarsi ma sempre verificarsi che un sistema possa penetrare all'interno del sistema: continuamente valutare l'accesso, limitare il movimento latera, verificare l'identità e il dispositivo.

5.3 Black Box Security

L'attaccante si assume di aver

5.4 Gray Box

L'attaccante si assume di avere:

- Accessi fisici parziali alle chiavi crittografiche come risultato di un canale che perde informazioni
  - Electromagnetic radiation analisys
  - Current/power consumption analisys

- Operation timing analisys

5.5 White box security

I sistemi di gestione di diritti digitiali sono considerati white box poiché:

- L'utente finale è capace di acquistare alcuni tipi di contenuti premium
- Il contenuto arriva all'utente crittato e decrittato solo quando viene guardato
- Un uutente malevolo può provare ad estrarre le chiavi crittografiche dalsoftware e usarle per ridistribuire il contenuto fuori dal sistema DMR.

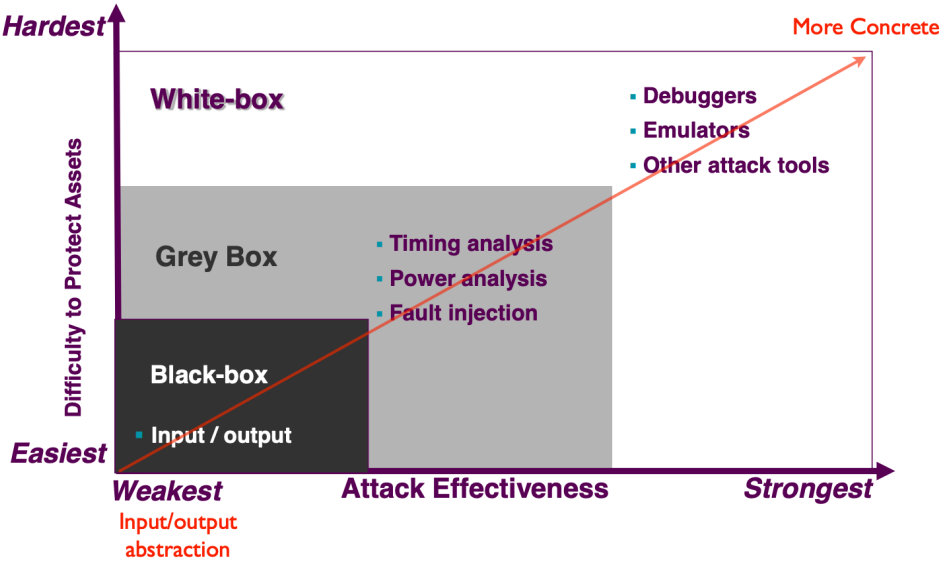


Figura 10: White box security

Il goal della white box criptography è quella di recuperare la chiave in uno scenario white box almeno difficile come una black-box security. Detti in altri modi, la pattern è quella di trasformare la chiave in modo tale che attaccare con un accesso white box non offre nessun vantaggio rispetto ad un accesso black box.

5.5.1 Code Obfuscation

La code obfuscation è una tecnica di white box crypto con l'obiettivo di rendere intelliggibile il codice sorgente o il bytecode di un programma, rendendo più difficile per un attaccante analizzare e comprendere il funzionamento del software.

Offuscare point function:

$$I_{x(w)} = \{1 \text{ if } w = x, 0 \text{ otherwise}\}$$

Offuscare I_x perfettamente con una *one-way function* f. Una one-way function è una funzione che è facile da calcolare in una direzione, ma difficile da invertire. Sia $y = f(x)$:

$$\text{Obf}I_{x(w)} = \{\text{if } y = f(w) \text{ then } 1, \text{otherwise } 0\}$$

intuitivamente $y = f(x)$ rivela poco o null rispetto all'accesso black box a I_x. Tutto ciò che può essere imparato al programma offuscato può essere imparato anche al programma non offuscato.